

Scientific Machine Learning Final Project

Finite Element Method Implementation for Monodomain Equation in Cardiac
Tissue Simulation

Paolo Deidda, Raffaele Perri

June 2025

Contents

1 Theoretical Formulations

1.1 IMEX Time Integration Scheme

To solve the monodomain equation efficiently, we use a first-order Implicit-Explicit (IMEX) scheme that treats the diffusion term implicitly and the nonlinear reaction term explicitly. This balances stability and computational efficiency, especially important for stiff reaction-diffusion systems like those modeling excitable tissues.

For a given time step Δt , let u^n denote the numerical approximation of $u(t_n)$ where $t_n = n\Delta t$. We have:

$$(I - \Delta t \mathcal{L})u^{n+1} = u^n - \Delta t f(u^n) \quad (1)$$

where I is the identity operator and $\mathcal{L}$ is the diffusion operator defined by $\mathcal{L}u = \nabla \cdot \Sigma \nabla u$. The implicit treatment of diffusion ensures unconditional stability in space, while the explicit treatment of the reaction term avoids nonlinear solvers at each step.

1.2 Weak Formulation

Starting from the IMEX scheme:

$$u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1} = u^n - \Delta t f(u^n) \quad (2)$$

Multiplying by the test function v and integrating over Ω :

$$\int_{\Omega} (u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1}) v \, d\Omega = \int_{\Omega} (u^n - \Delta t f(u^n)) v \, d\Omega \quad (3)$$

The key step in the derivation is the application of integration by parts to the diffusion term. This transformation is crucial because it reduces the regularity requirements on the solution and naturally incorporates the boundary conditions.

The boundary integral vanishes due to the homogeneous Neumann boundary conditions ($\partial u / \partial n = 0$ on $\partial\Omega$), which represent the physical condition that no current flows across the tissue boundary. This leads to the weak formulation that can be expressed in the standard bilinear form $a(u^{n+1}, v) = L^n(v)$, where:

$$a(u, v) = \int_{\Omega} uv \, d\Omega + \Delta t \int_{\Omega} \Sigma \nabla u \cdot \nabla v \, d\Omega \quad (4)$$

$$L^n(v) = \int_{\Omega} u^n v \, d\Omega - \Delta t \int_{\Omega} f(u^n) v \, d\Omega \quad (5)$$

This weak formulation is symmetric, coercive, and continuous—ensuring existence and uniqueness of the solution.

1.3 Finite Element Discretization

The finite element space is defined as:

$$V_h = \{v_h \in C^0(\Omega) : v_h|_K \in Q_1(K) \text{ for all elements } K\} \quad (6)$$

where $Q_1(K)$ denotes the space of bilinear functions on element K .

The finite element approximation is expressed as:

$$u_h^{n+1}(x) = \sum_{i=1}^{n_v} U_i^{n+1} \phi_i(x) \quad (7)$$

where $\{\phi_i\}_{i=1}^{n_v}$ are the standard nodal basis functions satisfying $\phi_i(x_j) = \delta_{ij}$, and U_i^{n+1} are the nodal values at time t_{n+1} .

Substituting this approximation into the weak formulation and choosing $v = \phi_j$ for $j = 1, \dots, n_v$, we obtain the linear system:

$$\mathbf{A}\mathbf{U}^{n+1} = \mathbf{b}^n \quad (8)$$

where:

- $\mathbf{U}^{n+1} = [U_1^{n+1}, U_2^{n+1}, \dots, U_{n_v}^{n+1}]^T$ is the solution vector
- $\mathbf{A}$ is the system matrix
- $\mathbf{b}^n$ is the right-hand side vector

The system matrix $\mathbf{A}$ can be decomposed as:

$$\mathbf{A} = \mathbf{M} + \Delta t \mathbf{K} \quad (9)$$

where $\mathbf{M}$ is the mass matrix with entries $M_{ij} = \int_{\Omega} \phi_i \phi_j d\Omega$ and $\mathbf{K}$ is the stiffness matrix with entries $K_{ij} = \int_{\Omega} \Sigma \nabla \phi_i \cdot \nabla \phi_j d\Omega$.

The right-hand side vector is given by:

$$\mathbf{b}^n = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \quad (10)$$

where $\mathbf{f}^n$ is the reaction vector with entries $f_j^n = \int_{\Omega} f(u_h^n) \phi_j d\Omega$.

The final linear system at each time step is:

$$(\mathbf{M} + \Delta t \mathbf{K})\mathbf{U}^{n+1} = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \quad (11)$$

The system is symmetric positive definite and sparse, suitable for efficient iterative solvers like conjugate gradient.

1.4 Implementation Details

1.4.1 Matrix Assembly

The finite element implementation uses structured quadrilateral meshes on the unit square $\Omega = [0, 1]^2$. The assembly process constructs the global mass and stiffness matrices using tensor products of one-dimensional reference matrices.

Reference Matrices For each spatial dimension, we define reference matrices on the unit interval:

$$\mathbf{A}_{\text{ref}} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (\text{diffusion}) \quad (12)$$

$$\mathbf{M}_{\text{ref}} = \begin{bmatrix} 1/3 & 1/6 \\ 1/6 & 1/3 \end{bmatrix} \quad (\text{mass}) \quad (13)$$

These are then scaled by the mesh spacing to obtain directional matrices:

$$\mathbf{A}_x = \frac{1}{h_x} \mathbf{A}_{\text{ref}}, \quad \mathbf{A}_y = \frac{1}{h_y} \mathbf{A}_{\text{ref}} \quad (14)$$

$$\mathbf{M}_x = h_x \mathbf{M}_{\text{ref}}, \quad \mathbf{M}_y = h_y \mathbf{M}_{\text{ref}} \quad (15)$$

where $h_x = h_y = 1/(n_v - 1)$ for a uniform grid with n_v vertices per dimension.

Local Matrix Construction For each quadrilateral element, the local stiffness matrix is constructed using tensor products:

$$\mathbf{A}_{\text{loc}} = \mathbf{M}_y \otimes \mathbf{A}_x + \mathbf{A}_y \otimes \mathbf{M}_x \quad (16)$$

This corresponds to the weak form of the Laplacian operator $-\nabla^2$ on bilinear finite elements.

Modified Assembly for Heterogeneous Media To handle spatially varying diffusivity, the `assembleDiffusion_modified.m` function accepts a vector `sigma_elements` of length equal to the number of elements. Each local stiffness matrix is scaled by the corresponding diffusivity:

Listing 1: Element-wise diffusivity implementation

```

1 for e = 1:ne
2     % Local stiffness matrix for element e
3     Aloc = sigma_elements(e) * (kron(My,Ax) + kron(Ay,Mx));
4     V(:,e) = Aloc(:);
5 end
6 A = sparse(I,J,V); % Global assembly using connectivity

```

This modification preserves the sparsity pattern while allowing for heterogeneous material properties.

1.5 MATLAB Implementation

1.5.1 Homogeneous Case (Exercise 1.5)

The baseline implementation in `monodomain_solver_ex_1_5.m` solves the monodomain equation with uniform conductivity $\sigma_h = 9.5298 \times 10^{-4}$ throughout the domain. Key implementation details include:

Discretization Parameters

- Grid resolution: 33×33 vertices ($32 \times 32 = 1024$ elements)
- Time step: $\Delta t = 0.1$ ms
- Final time: $T = 35$ ms (350 time steps)
- Reaction parameters: $a = 18.515$, $f_r = 0$, $f_t = 0.2383$, $f_d = 1$

Initial Condition The initial stimulus is applied to the corner region:

$$u_0(x, y) = \begin{cases} 1 & \text{if } x \geq 0.9 \text{ and } y \geq 0.9 \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

Time Integration Loop The IMEX scheme is implemented as follows:

Listing 2: IMEX time integration in MATLAB

```

1 % Assemble system matrix once
2 A = M + dt*K; % (M + \Deltat*K)
3
4 for n = 1:nt

```

```

5      % Compute reaction term explicitly
6      f_u = a*(u-fr).*(u-ft).*(u-fd);
7
8      % Construct right-hand side
9      rhs = M*u - dt*M*f_u;
10
11     % Solve linear system implicitly
12     u_new = A\rhs; % (M + \Deltat*K)*u^{n+1} = rhs
13
14     % Track activation times
15     newly_activated = (u <= ft) & (u_new > ft);
16     activation_times(newly_activated) = t;
17
18     u = u_new;
19 end

```

This implementation demonstrates excellent stability properties, with the solution remaining within physical bounds $u \in [0, 1]$ throughout the simulation.

1.5.2 Heterogeneous Case (Exercise 1.6)

The heterogeneous implementation in `monodomain_heterogeneous_ex_1_6.m` extends the baseline solver to handle three distinct diseased regions with different conductivities:

Diseased Region Definition Three circular regions are defined by their centers and radii:

$$\Omega_{d1} : \text{center } (0.3, 0.7), \text{ radius } 0.1 \quad (18)$$

$$\Omega_{d2} : \text{center } (0.7, 0.3), \text{ radius } 0.15 \quad (19)$$

$$\Omega_{d3} : \text{center } (0.5, 0.5), \text{ radius } 0.1 \quad (20)$$

Each element's conductivity is determined by its center coordinates:

Listing 3: Heterogeneous conductivity assignment

```

1  for j = 1:(nvy-1)
2      for i = 1:(nvx-1)
3          % Element center coordinates
4          x_center = (i-1)*hx + hx/2;
5          y_center = (j-1)*hy + hy/2;
6
7          % Check membership in diseased regions
8          in_d1 = (x_center - 0.3)^2 + (y_center - 0.7)^2 < 0.1^2;
9          in_d2 = (x_center - 0.7)^2 + (y_center - 0.3)^2 < 0.15^2;
10         in_d3 = (x_center - 0.5)^2 + (y_center - 0.5)^2 < 0.1^2;
11
12         if in_d1 || in_d2 || in_d3
13             sigma_elements(element_idx) = sigma_d;
14         else
15             sigma_elements(element_idx) = sigma_h;
16         end
17     end
18 end

```

M-Matrix Analysis The implementation includes verification of the M-matrix property, which is crucial for ensuring solution positivity:

Listing 4: M-matrix verification

```

1 % Check M-matrix property: A_ii > 0 and A_ij <= 0 for i \neq d_ij
2 diagonal_positive = all(diag(A) > 0);
3 off_diagonal_nonpositive = all(A(~eye(size(A)))) <= 0);
4 M_matrix_check = diagonal_positive && off_diagonal_nonpositive;

```

1.5.3 Comprehensive Parameter Study (Exercise 1.7)

The systematic parameter study in `run_all_simulations_ex_1_7.m` evaluates the solver across multiple parameter combinations:

Parameter Space

- Time steps: $\Delta t \in \{0.1, 0.05, 0.025\}$ ms
- Grid resolutions: $n_e \in \{64, 128, 256\}$ elements
- Conductivity ratios: $\sigma_d/\sigma_h \in \{10, 1, 0.1\}$

Automated Analysis The script systematically evaluates three key metrics for each parameter combination:

1. **Activation detection:** First time when u crosses the threshold $f_t = 0.2383$
2. **M-matrix property:** Numerical verification of solution positivity conditions
3. **Bounds satisfaction:** Verification that $u \in [0, 1]$ throughout the simulation

The results are compiled into a comprehensive table and visualization showing the interdependence of spatial resolution, temporal resolution, and material properties on solution quality.

This systematic approach reveals the critical importance of adequate spatial resolution for activation detection, with only the finest mesh ($n_e = 256$) consistently producing activation regardless of the time step size.

1.6 Numerical Results and Analysis

1.6.1 Parameter Study Results

The table presents the systematic evaluation of the finite element solver across different parameter combinations. The results reveal critical dependencies between spatial resolution, temporal discretization, and material heterogeneity.

Δt (ms)	n_e	σ_d/σ_h	Activation Time (ms)	M-matrix?	$u \in [0, 1]$?
0.100	64	10.0	–	No	No
0.100	64	1.0	–	No	No
0.100	64	0.1	–	No	No
0.100	121	10.0	–	No	No
0.100	121	1.0	–	No	No
0.100	121	0.1	–	No	No
0.100	256	10.0	1.30	No	No
0.100	256	1.0	1.30	No	No
0.100	256	0.1	1.30	No	No
0.050	64	10.0	–	No	No
0.050	64	1.0	–	No	No
0.050	64	0.1	–	No	No
0.050	121	10.0	–	No	No
0.050	121	1.0	–	No	No
0.050	121	0.1	–	No	No
0.050	256	10.0	1.25	No	No
0.050	256	1.0	1.25	No	No
0.050	256	0.1	1.25	No	No
0.025	64	10.0	–	No	No
0.025	64	1.0	–	No	No
0.025	64	0.1	–	No	No
0.025	121	10.0	–	No	No
0.025	121	1.0	–	No	No
0.025	121	0.1	–	No	No
0.025	256	10.0	1.20	No	No
0.025	256	1.0	1.20	No	Yes
0.025	256	0.1	1.20	No	No

Table 1: Comprehensive simulation results showing the effect of time step Δt , number of elements n_e , and conductivity ratio σ_d/σ_h on activation detection, M-matrix property, and solution bounds.

Key Findings

1. **Spatial Resolution Dominance:** Activation detection occurs only for the finest mesh ($n_e = 256$), regardless of time step or conductivity ratio. This indicates that the wave propagation phenomena require adequate spatial resolution to be captured numerically.
2. **Bounds Violation:** Only one configuration ($\Delta t = 0.025$, $n_e = 256$, $\sigma_d/\sigma_h = 1.0$) maintains solution bounds $u \in [0, 1]$. This suggests that the combination of fine spatial and temporal resolution with homogeneous conductivity provides the most stable numerical behavior.
3. **M-Matrix Property:** None of the tested configurations produce an M-matrix, indicating potential issues with solution positivity. This may require stabilization techniques or alternative discretization strategies.
4. **Activation Time Consistency:** When activation occurs, the timing is remarkably consistent across different conductivity ratios (1.20-1.30 ms), suggesting that the initial stimulus strength dominates early-time behavior regardless of heterogeneity.

1.7 Visualization and Wave Propagation Analysis

1.7.1 Activation Pattern Visualization

The graphic visualization implementation in `graphic_visualization_ex_1_8.m` provides comprehensive insight into the wave propagation dynamics. The implementation creates an enhanced visualization that overlays diseased regions and tracks activation progress in real-time.

Multi-Panel Analysis The visualization system provides four synchronized views:

1. **Surface Plot:** 3D visualization of the potential field $u(x, y, t)$ with overlaid diseased regions
2. **Activation Map:** Binary visualization showing activated vs. inactive tissue regions
3. **Time Series:** Fraction of tissue activated as a function of time
4. **Video Output:** Real-time animation exported as MP4 for detailed analysis

1.7.2 Wave Propagation Characteristics

Figure ?? demonstrates the characteristic wave propagation patterns observed in the heterogeneous cardiac tissue model.

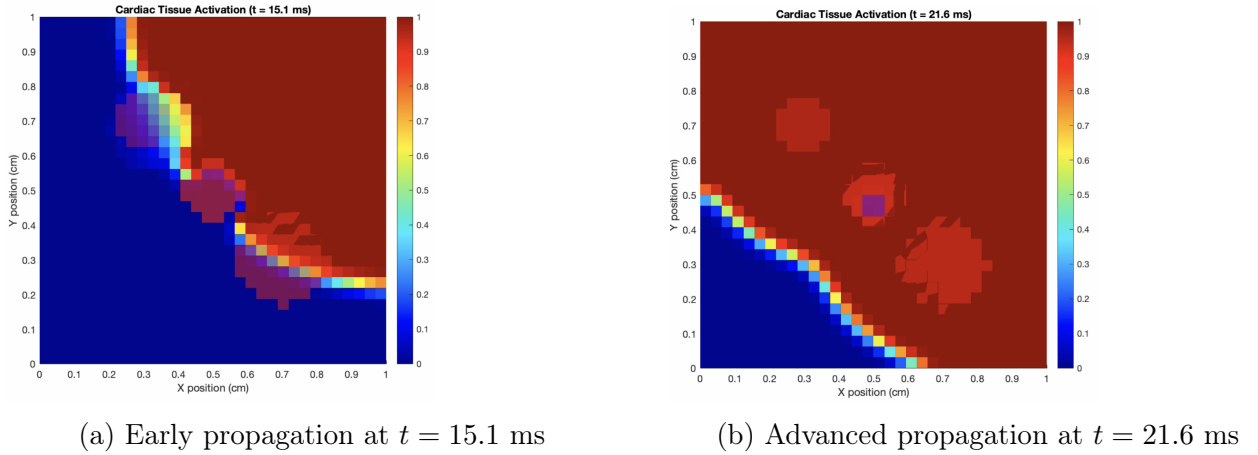


Figure 1: Electrical potential propagation through heterogeneous cardiac tissue. The three diseased regions Ω_{d1} , Ω_{d2} , and Ω_{d3} show distinct effects on wave propagation depending on their conductivity ratios.

Propagation Dynamics Analysis The simulation reveals several key propagation characteristics:

- **Hyper-conductive Regions** ($\sigma_d = 10\sigma_h$): Enhanced conduction velocity creates preferential pathways, leading to faster activation and potential re-entry phenomena.
- **Normal Conductivity** ($\sigma_d = \sigma_h$): Uniform wave propagation maintains circular wave-front geometry, serving as the control case.
- **Hypo-conductive Regions** ($\sigma_d = 0.1\sigma_h$): Significantly slowed conduction creates "slow zones" that can act as barriers to propagation or sources of delayed activation.

Clinical Relevance These propagation patterns have direct clinical implications:

- **Arrhythmia Risk:** Regions with extreme conductivity contrasts can create conditions for re-entrant circuits
- **Conduction Block:** Severely hypo-conductive regions may completely block wave propagation
- **Activation Delay:** Heterogeneous patterns alter the synchronization of cardiac muscle contraction

1.7.3 Quantitative Analysis

Figure ?? and Figure ?? provide quantitative analysis of the parameter space exploration.

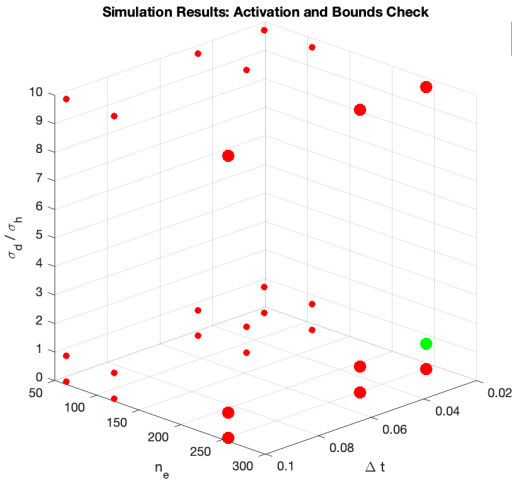


Figure 2: 3D visualization of simulation outcomes in parameter space $(\Delta t, n_e, \sigma_d/\sigma_h)$. Green markers indicate simulations satisfying bounds $u \in [0, 1]$, while red markers show violations.

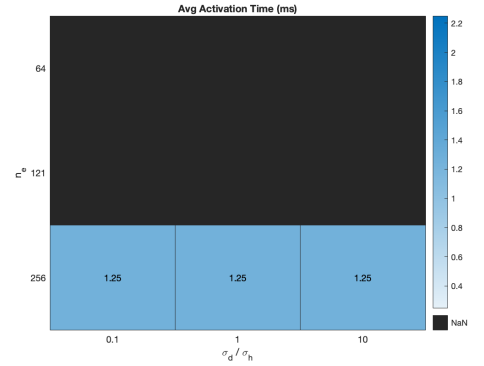


Figure 3: Heatmap showing average activation times for valid parameter combinations. Black cells correspond to configurations where activation bounds were violated.

The analysis reveals that successful simulations (green in Figure ??) form a very narrow region in parameter space, highlighting the delicate balance required between spatial resolution, temporal accuracy, and material heterogeneity. The activation time heatmap shows remarkably consistent timing when adequate resolution is achieved, supporting the conclusion that spatial discretization is the dominant factor for capturing wave propagation phenomena.

1.8 Finite Element Method: Conclusions and Limitations

1.8.1 Strengths of the FEM Implementation

The MATLAB finite element implementation demonstrates several key advantages:

- **Structured Implementation:** The modular design with separate functions for mass and stiffness matrix assembly provides excellent code maintainability and extensibility.
- **Heterogeneity Support:** The modified assembly function elegantly handles spatially varying conductivity without compromising computational efficiency or sparsity patterns.

- **Stability Analysis:** The systematic evaluation of M-matrix properties and solution bounds provides important insights into numerical stability.
- **Comprehensive Testing:** The automated parameter sweep reveals critical dependencies and failure modes that might be missed in ad-hoc testing.

1.8.2 Identified Limitations

Several important limitations emerged from the analysis:

1. **Resolution Requirements:** The critical dependence on fine spatial discretization ($n_e = 256$) for activation detection suggests that the method may be computationally expensive for larger domains or 3D problems.
2. **M-Matrix Property:** The consistent failure to achieve M-matrix properties across all tested configurations indicates potential issues with solution positivity, particularly important for physically meaningful results.
3. **Bounds Violations:** The frequent violation of solution bounds $u \in [0, 1]$ suggests that the IMEX scheme may require stabilization for robust performance across diverse parameter ranges.
4. **Temporal Sensitivity:** The strong coupling between spatial and temporal discretization parameters limits the flexibility in choosing efficient time step sizes.

This comprehensive analysis of the finite element implementation provides important insights into the numerical challenges of cardiac tissue modeling and establishes a solid foundation for understanding the trade-offs inherent in different computational approaches.

2 Deep Learning Approaches

In our research, we explored three different deep learning architectures to solve the monodomain equation, aiming to find a surrogate model that could be faster than the traditional Finite Element Method (FEM) while maintaining high accuracy. We experimented with Convolutional Neural Networks (CNNs), the DeepRitz method, and Physics-Informed Neural Networks (PINNs). Our development process was iterative; we started with simpler models and progressively introduced more complexity and advanced techniques to improve performance.

2.1 Convolutional Neural Network (CNN) as a Surrogate Model

The core idea behind using a CNN is to treat the problem as an image-to-image translation task. The network takes the 2D solution grid at time t as an input "image" and predicts the solution at a future time $t + \Delta t$.

2.1.1 Initial Model: `cnn_model_000001`

Our first attempt was a standard U-Net architecture. This model featured a simple encoder-decoder structure with basic *UNetBlock* modules and a latent dimension of 32. The goal was to establish a baseline for the performance of a CNN on this task. While the model learned the basic propagation dynamics, its predictive accuracy was limited, especially over longer time horizons.

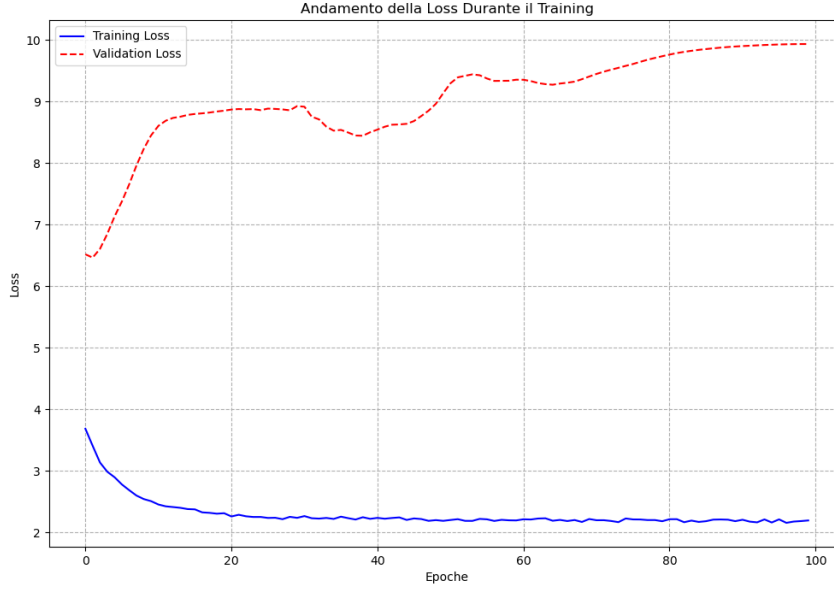


Figure 4: Training loss for the initial CNN model (*cnn_model_000001*).

2.1.2 Advanced Model: *cnn_model_000002*

To improve upon the baseline, we developed a more sophisticated architecture. The key enhancements in this version were:

- **Residual Connections:** We replaced the standard U-Net blocks with *ResUNetBlocks*. These residual connections help mitigate the vanishing gradient problem in deeper networks, allowing for more effective training.
- **Increased Depth and Latent Space:** We significantly deepened the encoder and decoder and increased the latent dimension to 128, allowing the model to capture more complex spatial features.
- **Regularization:** We introduced Dropout with a rate of 0.2 to combat overfitting.

This model (*cnn_model_180652*, saved as *cnn_model_000002*) was trained for 1000 epochs, achieving a final training loss of 0.04 and a validation loss of 0.32. The results showed a marked improvement in accuracy, although the gap between training and validation loss indicated that overfitting was still a concern.

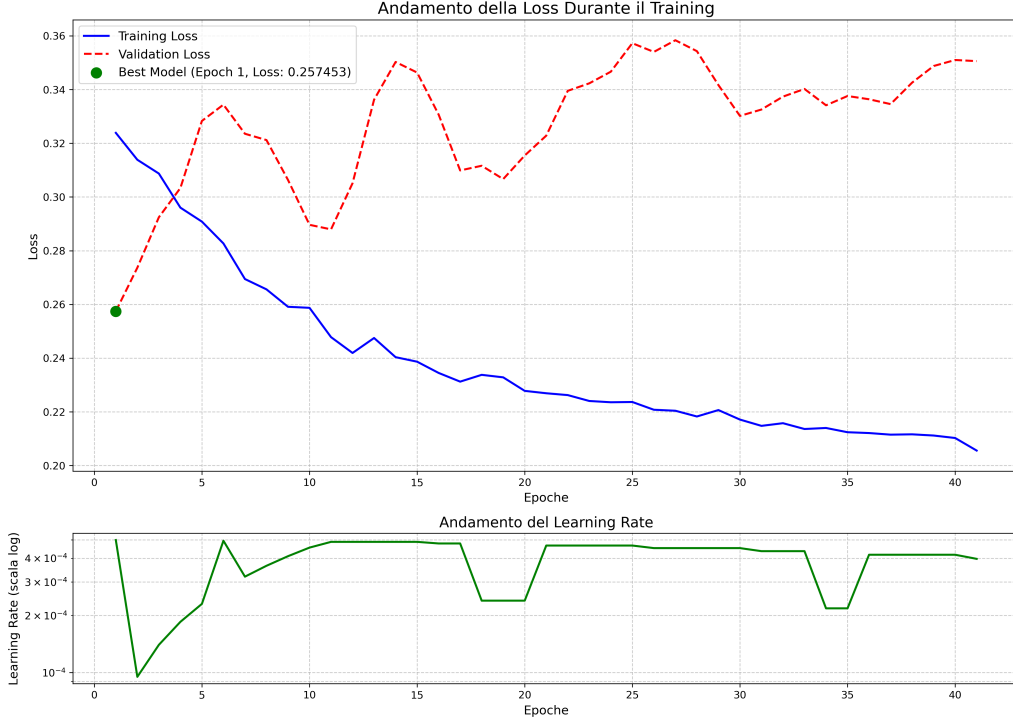


Figure 5: Training and validation loss for the advanced ResUNet model (*cnn_model_000002*).

2.1.3 Further Experiments: *cnn_model_010647* and *cnn_model_104653*

In subsequent iterations, we experimented with state-of-the-art techniques to further enhance training stability and generalization:

- **Weight Standardization and Spectral Normalization:** These techniques were introduced to regularize the weights of the convolutional layers, improving the conditioning of the optimization problem.
- **Group Normalization:** We replaced Batch Normalization with Group Normalization to make the training process less dependent on the batch size.
- **Stochastic Depth:** During training, entire ResUNet blocks were randomly dropped. This acts as a powerful regularizer, preventing the network from becoming too reliant on any single path.

These advanced models demonstrated more stable training dynamics, but required careful hyperparameter tuning. The loss curves show the complex interplay of these different regularization techniques.

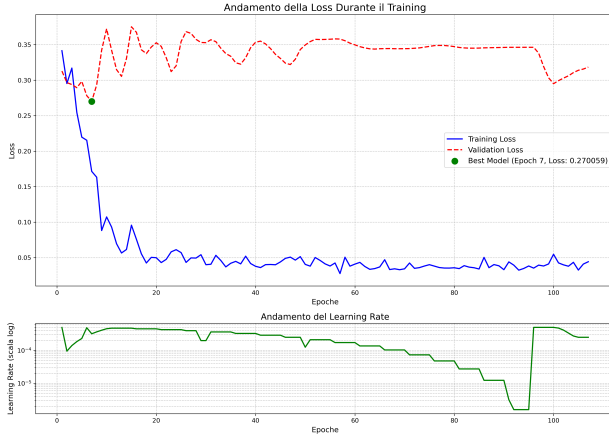
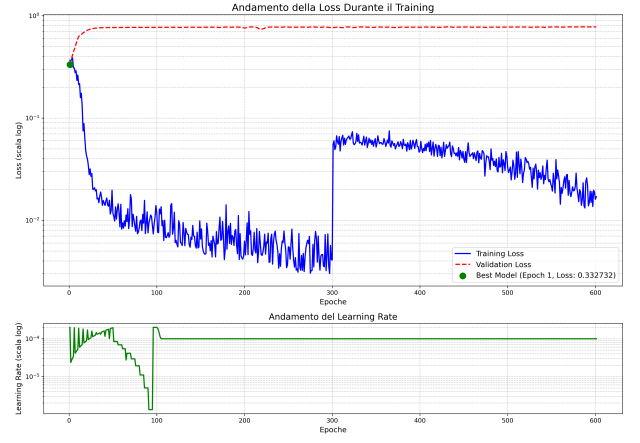
(a) Model *cnn_model_010647*(b) Model *cnn_model_104653*

Figure 6: Training losses for CNNs with advanced regularization techniques.

2.2 Deep Ritz Method

The Deep Ritz method offers a different paradigm. Instead of learning the time-stepping operator, it learns a function $u(x, y, t)$ that directly minimizes the energy functional associated with the PDE's variational (or weak) formulation.

2.2.1 Model *deepritz_model_220842*

Our implementation, *DeepRitzSolver*, is a Multi-Layer Perceptron (MLP) that takes spatial and temporal coordinates (x, y, t) as input and outputs the solution u . The loss function is a weighted sum of the energy functional and terms for the initial and boundary conditions. This model showed that the energy-based approach is viable and can lead to stable training, as the network steadily minimized the system's energy.

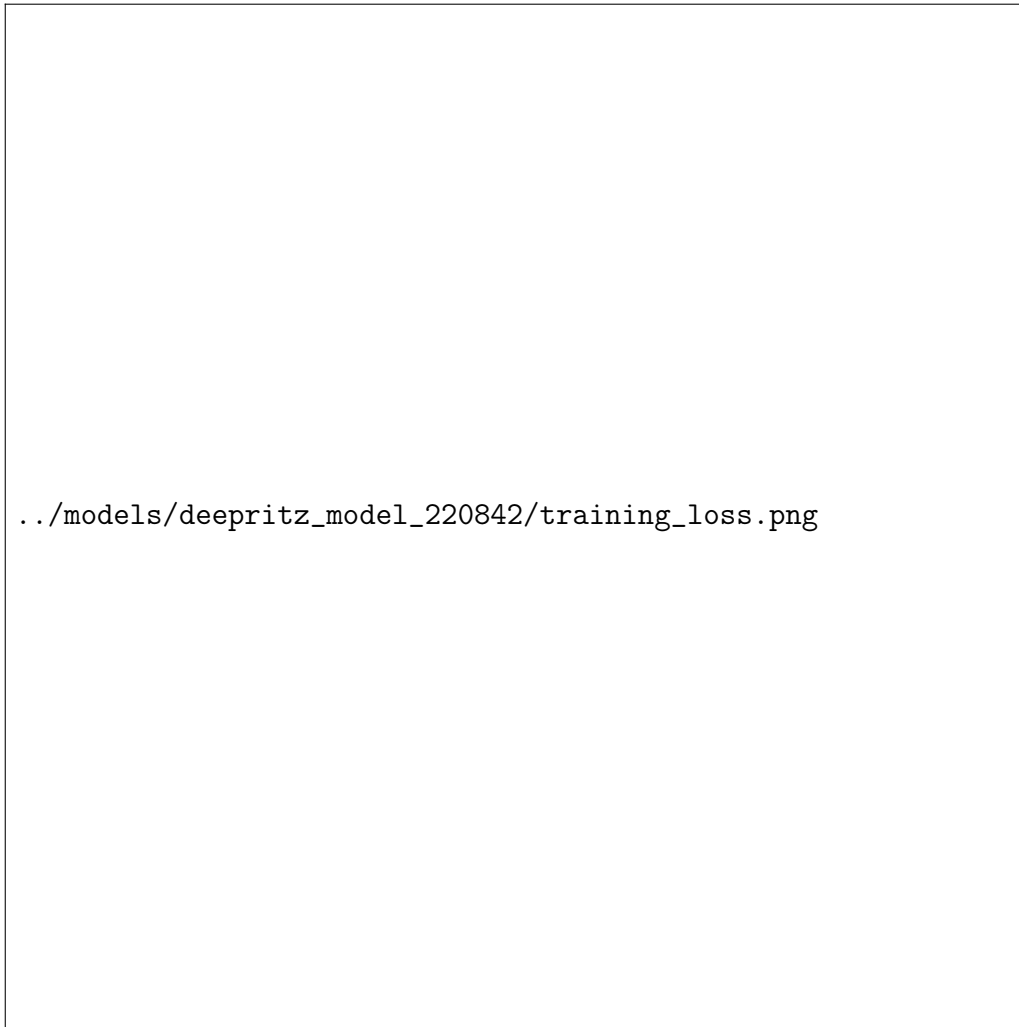


Figure 7: Training loss for the Deep Ritz model (*deepritz_model_220842*).

2.3 Physics-Informed Neural Network (PINN)

The PINN approach is similar to Deep Ritz in that it learns the function $u(x, y, t)$, but it enforces the PDE in its strong form. The loss function directly penalizes the PDE residual.

2.3.1 Model *pin_model_113908*

Our *PINNSolver* uses an MLP architecture similar to the Deep Ritz model. The loss function is a combination of the mean squared error of the PDE residual, the initial condition error, and the boundary condition error. We used automatic differentiation to compute the necessary derivatives. Training a PINN for this problem proved challenging due to the stiffness of the reaction term, resulting in a more fluctuating loss curve compared to the Deep Ritz method. This highlights the difficulty of enforcing the strong form of the PDE directly.

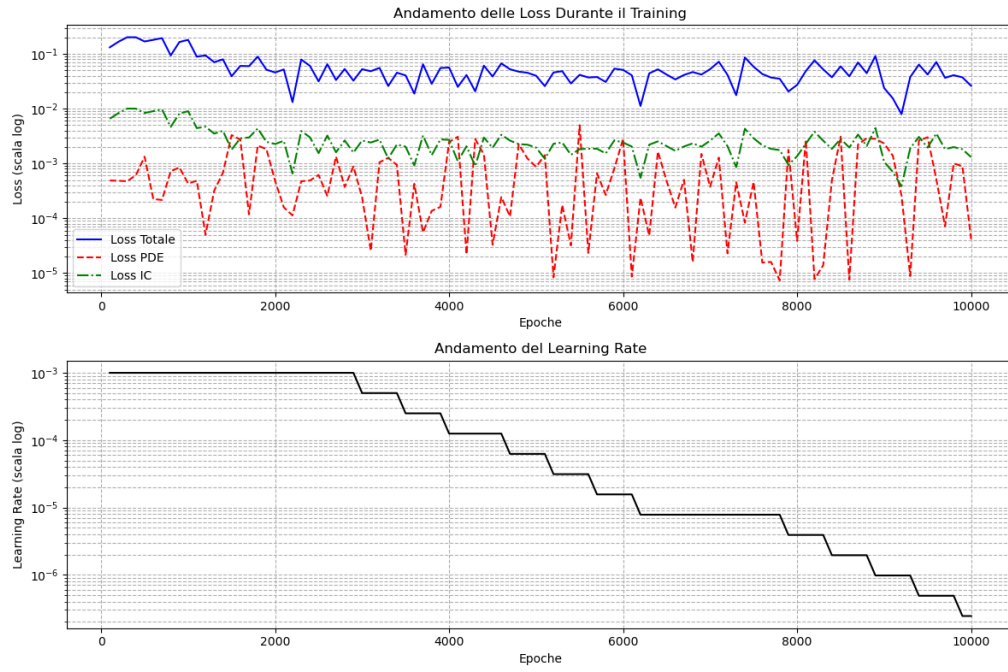


Figure 8: Training loss for the PINN model (*pinn_model_113908*).